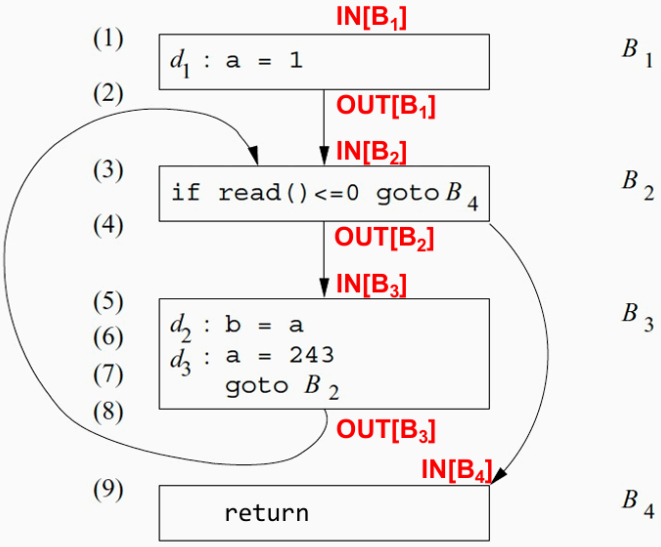
Recap-2
The Compilers' Middle End

Data Flow Analysis



- IN[B]/OUT[B]
  - Set of data flow facts before and after a block
- Constraints
  - OUT[B] = f_B(IN[B])
  - IN[B] = $\wedge_{P \in preds(B)}$ OUT[P]

Transfer & Merging Functions

- Transfer Functions?
- Merge Functions?
- Forward or Backward Analysis?

Data Flow Analysis

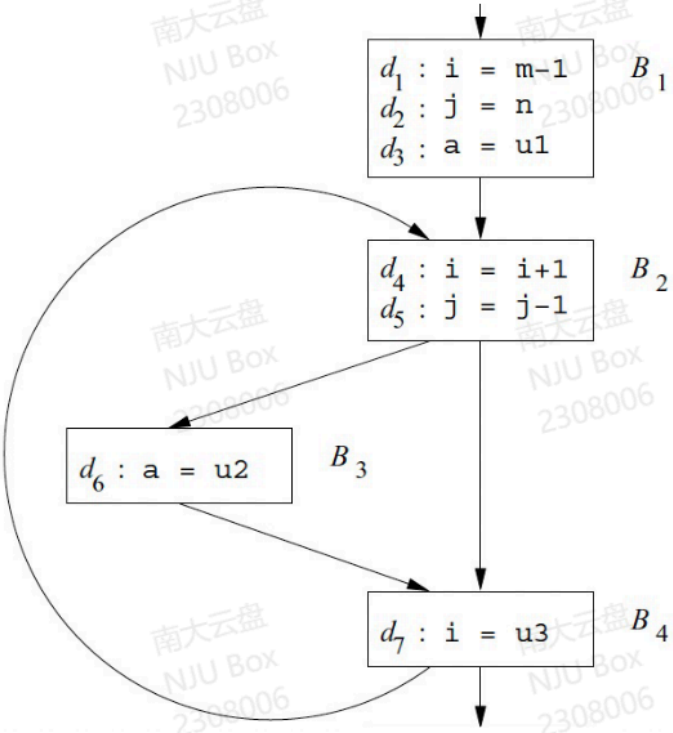
- To perform data flow analysis, define
  - transfer function f for each statement/block
  - merge function $\wedge$ at the joint point of multiple paths
  - the initial set of data flow facts for each block

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

- **While** (!worklist.empty()) **Do**
- B = worklist.pop();
- IN[B] = $\wedge_{P \in preds(B)}$ OUT[P]; OUT[B] = f_B(IN[B])
- **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Reaching Definitions



$gen_{B_1} = \{ d_1, d_2, d_3 \}$

$kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$

$gen_{B_2} = \{ d_4, d_5 \}$

$kill_{B_2} = \{ d_1, d_2, d_7 \}$

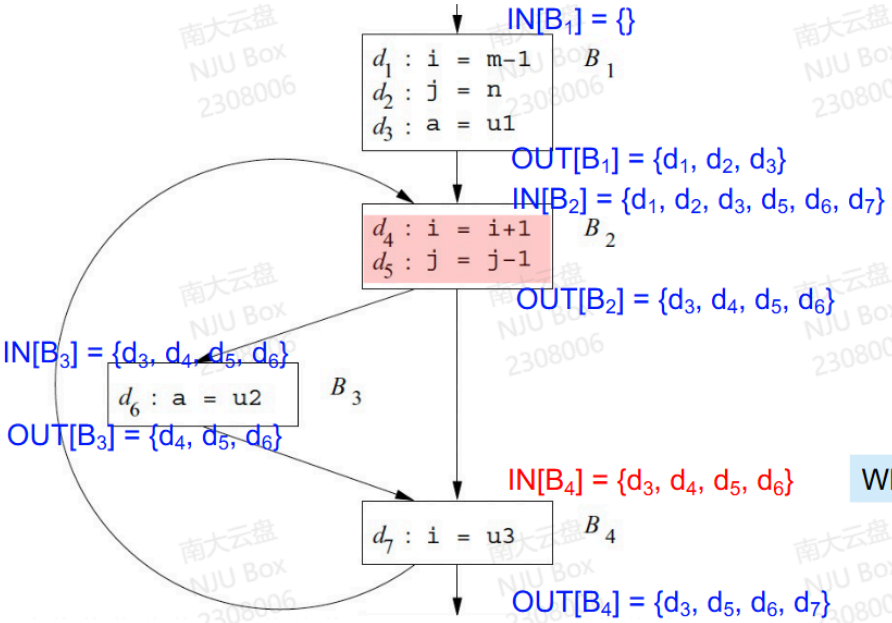
$gen_{B_3} = \{ d_6 \}$

$kill_{B_3} = \{ d_3 \}$

$gen_{B_4} = \{ d_7 \}$

$kill_{B_4} = \{ d_1, d_4 \}$

Reaching Definitions



Why cannot d_1, d_2, d_7 reach here?

Available Expressions

- An expression x + y is available at a point p if
  - every path from the entry node to p evaluates x + y, and
  - after the last such evaluation prior to reaching p, there are no subsequent assignments to x or y.

- The equations

$$\boxed{\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)}$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
`a = b + c`	
	$\{b+c\}$
`b = a - d`	
	$\{a-d\}$
`c = b + c`	
	$\{a-d\}$
`d = a - d`	
	$\emptyset$

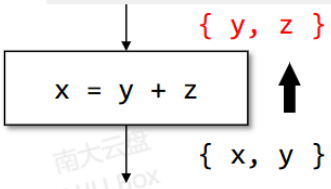
Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p.
- **Backward data flow analysis.**
  - The same as forward analysis except for the direction.

$$\boxed{\text{IN}[B] = use_B \cup (\text{OUT}[B] - def_B)}$$

$$\text{OUT}[B] = \bigcup_{S \text{ a successor of } B} \text{IN}[S]$$

The value of x at this point is dead after the definition in the block.

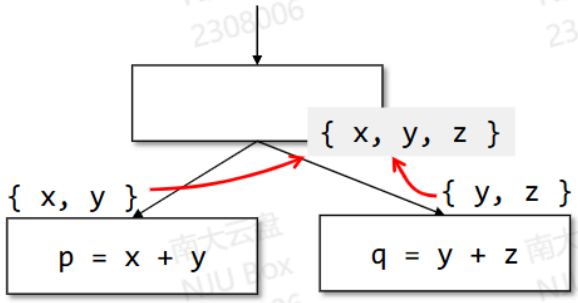


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p.
- **Backward data flow analysis.**
  - The same as forward analysis except for the direction.

$$\text{IN}[B] = use_B \cup (\text{OUT}[B] - def_B)$$

$$\boxed{\text{OUT}[B] = \bigcup_{S \text{ a successor of } B} \text{IN}[S]}$$



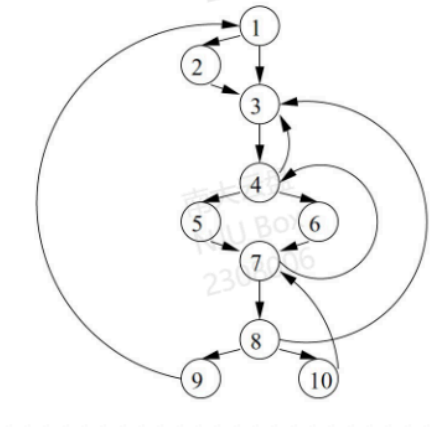
Question-I

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

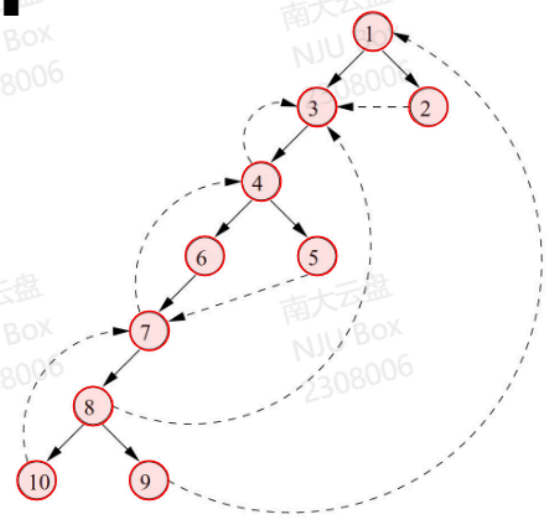
What's the best order of blocks?

- **While** (!worklist.empty()) **Do**
- B = worklist.pop();
- IN[B] = $\wedge_{P \in preds(B)}$ OUT[P]; OUT[B] = f_B(IN[B])
- **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Depth-First Order



Control-Flow Graph



Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Post-Order of the Tree: 10, 9, 8, 7, 6, 5, 4, 3, 2,1

Depth-First Order: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

请问这个satisfiabilty和这个symbolic execution与这个流敏感是什么关系

Symbolic Execution

```
1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
```

σ = { a → α_a ; b → α_b ; x → 1 ; y → 0 };
π = *true*

σ = { a → α_a ; b → α_b ; x → 1 ; y → 0};
π = $\alpha_a = 0$
check: $\pi \wedge \neg(1 - 0 \neq 0)$

σ: symbolic store that maps each variable to symbolic expressions.
π: path constraints. Initially, π = *true*.

Symbolic Execution

```
1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
```

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0\}$;
$\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 4\}$;
$\pi = \alpha_a \neq 0$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 2(\alpha_a + \alpha_b) ; y \rightarrow 4\}$;
$\pi = \alpha_a \neq 0 \wedge \alpha_b = 0$

check: $\pi \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$

σ: symbolic store that maps each variable to symbolic expressions.
π: path constraints. Initially, π = *true*.

$$\pi \cap \neg p$$

- Constraint solver
  - Satisfiable => A possible solution
  - Unsatisfiable
  - Unknown (due to timeout)

只有当它可能是true的时候，才有可能会检测出来问题，如果是false 要么就是到达不了这里，要么就是后面的条件是正确的，根本asserterror根本不可能出现

符号执行（Symbolic Execution）是一种程序分析技术，它解决的核心问题是：**当程序的输入存在不确定性或需要探索多种可能路径时，如何高效、系统地理解程序的行为并发现潜在问题（如错误、漏洞）。**

- Path Explosion
- Constraint Solving
- External Function Call

面临的挑战：

- **路径爆炸：** 程序路径数量随分支和循环次数指数级增长。
- **约束求解：** 复杂的非线性约束、浮点数约束、调用外部库函数等可能超出求解器能力。

Propositional Logic (PL)

- **Syntax of PL**, i.e., what are legal sentences of PL?
- (1) **Truth Symbols (b)**: $\top$ and $\bot$ (true and false)
- (2) **Propositional Variables (v)**: $p,\ q,\ r,\ \ldots$

  Atoms
- (3) **Literals (l)**: atom or its negation
- (4) **Formulas (F)**: $l \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2$

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
  - (1) **Truth Symbols** (b): $\top$ and $\bot$ (true and false)
  - (2) **Propositional Variables** (v): p, q, r, …
  - (3) **Literals** (l): atom or its negation
  - (4) **Formulas** (F): $l \mid \cancel{\neg F} \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid \cancel{F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2}$

- Which of these formulas are in NNF?
  - $(p \wedge q) \Rightarrow (p \vee \neg q)$
  - $\neg p \vee \neg q \vee p \vee \neg q$
  - $\neg(p \wedge q) \vee p \vee \neg q$

Theorem [Sufficiency of NNF]
For all formulae F in propositional logic, there exists an equivalent formula in negation normal form.

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

- Which of these formulas are in DNF?
  - $(p \wedge q) \Rightarrow (p \vee \neg q)$
  - $\neg p \wedge \neg q \vee (p \wedge \neg q)$
  - $\neg(p \wedge q) \vee p \vee \neg q$
  - $\neg p \vee \neg q \vee p \vee \neg q$

Theorem [Sufficiency of DNF]
For all formulae F in PL, there exists an equivalent formula in DNF.

Conjunctive Normal Form (CNF)

- Formulas in CNF are conjunctions of disjunctive literals

$$\bigwedge_i \bigvee_j l_{ij}$$

- Which of these formulas are in CNF?
  - $(p \wedge q) \Rightarrow (p \vee \neg q)$
  - $(\neg p \vee \neg q) \wedge (p \vee \neg q)$
  - $\neg p \vee \neg q \vee p \vee \neg q$

Theorem [Sufficiency of CNF]
For all formulae F in PL, there exists an equivalent formula in CNF.

Tseitin Transformation

- $\underbrace{\underbrace{(r \Rightarrow p)}_{x_2} \Rightarrow \overbrace{(\underbrace{\neg(\overbrace{q \wedge r}^{x_5})}_{x_4} \Rightarrow p)}^{x_3}}_{x_1}$

$$\begin{aligned} & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\ \wedge\ & x_2 \Leftrightarrow (r \Rightarrow p) \\ \wedge\ & x_3 \Leftrightarrow (x_4 \Rightarrow p) \\ \wedge\ & x_4 \Leftrightarrow \neg x_5 \\ \wedge\ & x_5 \Leftrightarrow q \wedge r \end{aligned}$$

Theorem [Efficient CNF Encoding]
For all formulae F in PL, there exists an equisatisfiable formula in CNF that is at most a constant factor larger than F.

Tseitin Transformation（泽廷变换） 是一种用于**将任意布尔逻辑公式（Boolean Formula）高效地转换成等价的可满足性保持（Equisatisfiable）的合取范式（Conjunctive Normal Form, CNF）** 的核心算法

问题：如何高效地将复杂逻辑结构转换为适合SAT求解器处理的CNF格式？

- **背景：**
    1. **SAT求解器的威力：** 布尔可满足性问题（SAT）是NP-完全问题，但现代高效的SAT求解器（如基于DPLL/CDCL算法，如MiniSAT, Glucose, Z3）能处理数百万变量和子句的工业级问题。这些求解器几乎只接受输入公式必须是**CNF形式**。
    2. **CNF的优势：** CNF是形如（A ∨ B） ∧ （¬C ∨ D ∨ E） ∧ ... 的公式集合（子句的与）。其结构简单、统一，利于SAT求解器应用高效的消解（Resolution）、单元传播（Unit Propagation）、子句学习（Clause Learning）等规则。
    3. 现实挑战：

       实际问题中需要表达的公式通常结构复杂：
        - 包含深度嵌套的逻辑门（如 A → （B ∧ C））。
        - 使用非CNF操作符（如异或 ⊕）。
        - 有大量共享子表达式（会导致传统“分配律展开法”产生指数级膨胀）。
- **传统方法的缺陷（分配律展开）：**
    - 直接使用逻辑定律（如德·摩根律、分配律）递归地将非CNF公式（如蕴含 →、异或⊕、非否定范式的与或非组合）强行展开成CNF。
    - **致命缺陷：** 对于具有大量共享子表达式或深层嵌套结构的公式，这种展开会导致**公式大小的指数级膨胀**。这使得转换后的CNF变得巨大甚至无法处理。

Tseitin Transformation 的解决方案：

泽廷变换**克服了传统方法的指数膨胀问题**，它通过**引入辅助变量（Auxiliary Variables）**，将一个逻辑电路的输出（或复杂公式的子结构）线性的（线性大小增长）、结构化的转换为CNF。其核心思想是：

1. **逻辑电路视角：** 将目标公式视为一个**布尔电路（Boolean Circuit）**，其中内部节点代表逻辑门（NOT, AND, OR, XOR 等），叶子节点是原始变量。
2. 引入辅助变量：
    - 为每个子表达式（尤其是内部逻辑门的输出）**引入一个新的辅助变量**（如 G_1, G_2, ..., Gn）。
    - 该辅助变量**代表其对应子表达式的真值**。
3. 定义辅助变量：
    - **关键步骤：** 为每个引入辅助变量 G 所代表的逻辑门（或子表达式），编写一组小的 **CNF子句** 来**精确等价地定义** G 和其输入之间的逻辑关系。
4. 整体CNF结构：
    - 将整个目标公式 F 的顶层输出也看作一个新的辅助变量（通常记作 T）。
    - 最终转换后的CNF公式由两部分组成：

- **(a)** 所有定义辅助变量的子句集合（Clauses defining auxiliary variables）。
- **(b)** 一个**强制顶层输出 T 必须为真** 的**单子句**：（T）。

转换过程直观示例（核心思想）：

假设要转换一个简单公式：F = A → （B ∧ C） （等价于 ¬A ∨ （B ∧ C））。

1. **识别结构 (电路)：**

   markdown

   复制

```
      → (F)
    /    \
  A      AND
        /   \
      B      C
```

2. **引入辅助变量：**
   - 为 AND 门引入辅助变量 G_1 代表 B ∧ C。
   - 为 → 门引入辅助变量 G_2 代表 A → G_1 (等价于 F)。
   - 最终目标：保证 G_2 为真（因为 F 就是 G_2）。
3. **定义辅助变量关系（用等价 CNF 表示）：**
   - 定义 G_1 (代表 B ∧ C):
     - G_1 为真 **当且仅当** B **和** C 都为真。
     - 这个关系可编码成 4 个子句（标准 AND 门 CNF 定义）：

        - $(\neg G_1 \lor B)$ // 如果 G_1 真，则 B 必须真
        - $(\neg G_1 \lor C)$ // 如果 G_1 真，则 C 必须真
        - $(G_1 \lor \neg B \lor \neg C)$ // 如果 B 和 C 不全真，则 G_1 必须假
        - **更精炼的 3 子句：** 实际通常只用 3 个子句即可完备定义（$(\neg G_1 \lor B) \land (\neg G_1 \lor C) \land (G_1 \lor \neg B \lor \neg C)$ 或等价形式）
    - 定义 G_2 (代表 $A \to G_1$，等价于 $\neg A \lor G_1$):
        - G_2 为真 **当且仅当** $\neg A \lor G_1$ 为真（即蕴含关系）。
        - 这个关系通常编码成 2 个子句：
            - $(\neg G_2 \lor \neg A \lor G_1)$ // 这是 $G_2 \to (\neg A \lor G_1)$ 的等价形式（分解蕴涵），但常简化为：
            - 常用单子句：

```
(¬A ∨ G₁ ∨ ¬G₂)
```

？实际更常见直接将蕴涵关系定义为 1 个子句

```
(¬G₂ ∨ ¬A ∨ G₁)
```

。标准做法是：

    - $(G_2 \lor A)$ // 如果 G_2 假，则 A 必须真（蕴含的前件真而结果假时才可能假）
    - $(G_2 \lor \neg G_1)$ // 如果 G_2 假，则 G_1 必须假
    - $(\neg G_2 \lor \neg A \lor G_1)$ // 这是最直观的蕴涵定义：G_2 蕴含 $\neg A \lor G_1$（即 $(A \land \neg G_1) \to \neg G_2$ 的逆否命题）
- **更高效的1子句：** 通常 $A \to B$ 可以直接等价定义为 $(\neg A \lor B)$，这里 G_2 代表 $A \to G_1$，所以可以直接写 $G_2 \equiv (\neg A \lor G_1)$。
- 定义 $G_2 \equiv (\neg A \lor G_1)$ 的标准CNF（2个子句）：
    - $(\neg G_2 \lor \neg A \lor G_1)$ // G_2 真蕴含 $\neg A \lor G_1$ 真
    - $(G_2 \lor A) \land (G_2 \lor \neg G_1)$ // G_2 假蕴含 A 真且 G_1 假（即 $\neg A \lor G_1$ 假）
    - 或者更常用 **$(\neg G_2 \lor \neg A \lor G_1) \land (G_2 \lor A) \land (G_2 \lor \neg G_1)$ 共3子句？** 实际 **只需 $(\neg G_2 \lor \neg A \lor G_1)$ 这1个子句** 并不完备！漏掉了当 A 假时 G_2 可以为真也可以为假的情况。标准完备定义是：
    - G_2 等价于 $\neg A \lor G_1$ 等价于 $(\neg A) \lor G_1$，写为CNF只需要 **1个子句：$(\neg G_2 \lor \neg A \lor G_1)$ 并不代表完备**。正确的做法是认识到 $G_2 \equiv (\neg A \lor G_1)$

本身就是一个子句！ 所以通常直接定义 $(\neg A \lor G_1 \lor \neg G_2)$？ 错误。

- 正确高效的表示 (1个子句)：

  定义

  ```
  G₂ ≡ (¬A ∨ G₁)
  ```

  的

  完全等价

  CNF 约束是：

  - $(G_2 \lor A) \land (G_2 \lor \neg G_1) \land (\neg G_2 \lor \neg A \lor G_1)$ -> 这太冗余。

- 更标准做法：

  直接将

  ```
  G₂
  ```

  定义为

  ```
  A → G₁
  ```

  的关系，常用 2 个子句或利用

```
≡
```

定义：

markdown

复制

```
// 使用等价式定义（需要4个子句，更高效）
// 等价形式 G₂ ↔ (¬A ∨ G₁) 可展开为：
// (G₂ → (¬A ∨ G₁)) ∧ ((¬A ∨ G₁) → G₂)
// 第一部分 (G₂ → (¬A ∨ G₁)): (¬G₂ ∨ ¬A ∨ G₁)  [1个子句]
// 第二部分 ((¬A ∨ G₁) → G₂): 等价于 ¬(¬A ∨ G₁) ∨ G₂ 等价于 (A ∧ ¬G₁) ∨ G₂ 等价于 (A ∨ G₂) ∧ (¬G₁ ∨ G₂) [2个子句]
```

    - **共需要 3 个子句：(¬G_2 ∨ ¬A ∨ G_1) ∧ (G_2 ∨ A) ∧ (G_2 ∨ ¬G_1)** -> 这是最完备的。
  - **实际简化：** 在许多实现和理论中，因为顶层需要让 G_2=True，只要保证 (¬G_2 ∨ ¬A ∨ G_1) 这个约束存在，当 G_2=True时它就会强制 ¬A ∨ G_1=True，所以有时可以省略另外两个约束（如果求解器设置 `T=true` 单子句）。但为了通用性，通常定义任何门都需用完备的等价CNF定义。

4. **顶层输出约束：**
    - 添加一个单子句 (G_2) 表示要求整个公式 F（即 G_2）必须为真。
5. **最终转换后的CNF公式：**
    - **原始变量：** A, B, C
    - **辅助变量：** G_1 (代表 B ∧ C), G_2 (代表 A → G_1)

- **CNF子句：**

```
// 定义 G₁ (B ∧ C)  [通常3子句]
(¬G₁ ∨ B)          // G₁ => B
(¬G₁ ∨ C)          // G₁ => C
(G₁ ∨ ¬B ∨ ¬C)     // ~(B ∧ C) => ~G₁ 或 G₁ ∨ ¬B ∨ ¬C

// 定义 G₂ (A → G₁) ≡ (¬A ∨ G₁) [等效2子句更常见或上面3子句]
// 最完备：3子句（当该门非顶层）
(¬G₂ ∨ ¬A ∨ G₁)   // G₂ => (¬A ∨ G₁)
(G₂ ∨ A)           // ~G₂ => A    (等价于 (¬A → G₂) 分解?)
(G₂ ∨ ¬G₁)         // ~G₂ => ¬G₁ (等价于 (G₁ → G₂))

// 更高效常用：因 A→G₁ 是蕴涵，有时简化为2子句（但前提是明确其结构）：
// 常用形式：仅使用 `(¬G₂ ∨ ¬A ∨ G₁)` 并结合顶层约束，但可能定义不完备。
// 更推荐等价写法：定义 G₂ ≡ (¬A ∨ G₁) 用4子句太冗余，实际标准只用：
    (¬G₂ ∨ ¬A ∨ G₁) ∧ (G₂ ∨ A) ∧ (G₂ ∨ ¬G₁)   // 3子句

// 顶层输出约束：
(G₂) // 整个公式 F (即 G₂) 必须为真
```

- **实际转换结果通常更精简（如借助定义门的标准 CNF 模板）：**

```
// 定义 AND 门 G₁ = B ∧ C:
    c1: (¬G₁ ∨ B)
    c2: (¬G₁ ∨ C)
    c3: (G₁ ∨ ¬B ∨ ¬C)

// 定义 OR 门 G₂ = (¬A ∨ G₁)  [注意 G₂ = (¬A ∨ G₁) 是一个OR门：输入¬A和G₁]
```

```
    c4: (¬G₂ ∨ ¬A ∨ G₁)  // OR门的标准定义之一(冗余但有效)
   // 更标准的OR定义 (G₂ = X ∨ Y, 这里X=¬A, Y=G₁) 需要额外处理输入为文字的情况。一般用:
    c4: (G₂ ∨ A)        // 从OR定义推导来的 (参见标准OR-CNF定义)
    c5: (G₂ ∨ ¬G₁)       // OR定义另一部分
    c6: (¬G₂ ∨ ¬A ∨ G₁) // OR定义的核心约束

// 但更常见的是采用直接定义等价门的简洁CNF。实践中 `A → G₁` 可直接等效为 `(¬A ∨ G₁)` 这个文字构成的子句（不需要引入G₂）？ 不对，这里引入G₂就是为了避免结构复杂。在高层视角:

  引入 G₂ = ¬A ∨ G₁   -> 这就变成了一个OR门（输入是¬A和G₁两个文字）
  一个两输入的OR门 G₂ = X ∨ Y (X=¬A, Y=G₁) 的CNF定义为 3个子句:
    (¬G₂ ∨ X ∨ Y)   -> (¬G₂ ∨ ¬A ∨ G₁)  [子句1]
    (G₂ ∨ ¬X)       -> (G₂ ∨ A)         [因为X=¬A, 所以¬X = A]
    (G₂ ∨ ¬Y)       -> (G₂ ∨ ¬G₁)       [子句2]

// 因此定义 G₂ 需 3 子句:
   c4: (¬G₂ ∨ ¬A ∨ G₁)
   c5: (G₂ ∨ A)
   c6: (G₂ ∨ ¬G₁)

// 顶层:
   c7: (G₂)

// 总计 7 个子句（c1~c7），变量数: 原始3个 (A,B,C) + 辅助2个 (G₁,G₂) = 5个变量。
```

Satisfiability Modulo Theories

- For the theory of bit vectors
  - **Step 1**: Word-Level Preprocessing
  - **Step 2**: Bit-Blasting (From FOL to PL)
  - **Step 3**: DPLL/CDCL

Example of Step 1:

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0) \xrightarrow{\text{Step 1}} \alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \alpha_a = 2$$

$$\xrightarrow{\text{Step 1}} \alpha_b = 0 \wedge \alpha_a = 2$$

- A first-order language is defined by
  - (1) Symbols for objects, e.g., $a, b, c, \ldots$
  - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \ldots, t_n)$
  - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \ldots, t_n)$

  Terms (t)

  - (4) Formula (F): $\top \mid \bot \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x.F \mid \exists x.F$

Satisfiability Modulo Theories

- For the theory of bit vectors
  - **Step 1**: Word-Level Preprocessing
  - **Step 2**: Bit-Blasting (From FOL to PL)
  - **Step 3**: DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \ldots, a_0], y = [b_7, b_6, \ldots, b_0]$ and $z = [c_7, c_6, \ldots, c_0]$ are three 8bit bit vectors

$$x \mid y = z \xrightarrow{\text{Step 2}} \bigwedge_{i=0}^{7} ((a_i \vee b_i) \Leftrightarrow c_i)$$

位向量每一位进行单独的运算就可以变成PL的CNF的形式

Tseitin Transformation 用在CNF解决size explosion的问题，可以使得CNF不用有size 爆炸的忧虑

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

- **Goal**: Find a solution to satisfy the input formula

- Repeat the following steps:
- (1) Choose a variable, p, assign true or false to the variable
- (2) Propagate the value of p

Theorem [Resolution]

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee c) \wedge (b_0 \vee b_1 \vee \cdots \vee b_m \vee \neg c)$$

can be simplified into

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee b_0 \vee b_1 \vee \cdots \vee b_m)$$

Andersen's Algorithm

Inclusion-based pointer analysis

Statement	Constraint	Shorthand
y = &x	pts(y) ⊇ {x}	
y = x	pts(y) ⊇ pts(x)	
*y = x	∀ v ∈ pts(y): pts(v) ⊇ pts(x)	pts(*y) ⊇ pts(x)
y = *x	∀ v ∈ pts(x): pts(y) ⊇ pts(v)	pts(y) ⊇ pts(*x)

Steensgaard's Algorithm

Inclusion-based pointer analysis (Andersen's Algorithm)

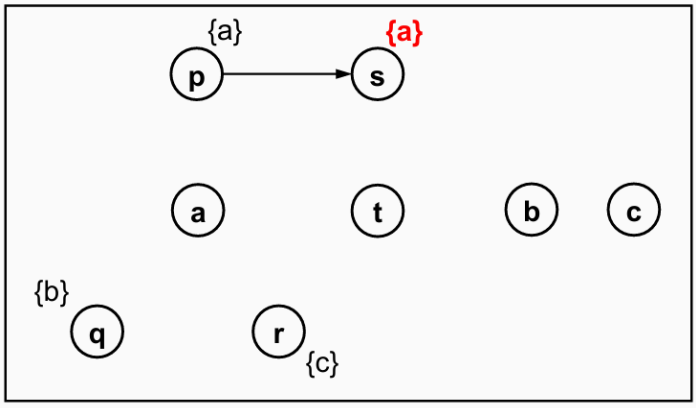
Statement	Constraint	Shorthand
y = &x	pts(y) ⊇ {x}	
y = x	pts(y) ⊇ pts(x)	
*y = x	∀ v ∈ pts(y): pts(v) ⊇ pts(x)	pts(*y) ⊇ pts(x)
y = *x	∀ v ∈ pts(x): pts(y) ⊇ pts(v)	pts(y) ⊇ pts(*x)

Unification-based pointer analysis (Steensgaard's Algorithm)

Statement	Constraint	Shorthand
y = &x	pts(y) ⊇ {x}	
y = x	pts(y) = pts(x)	
*y = x	∀ v ∈ pts(y): pts(v) = pts(x)	pts(*y) = pts(x)
y = *x	∀ v ∈ pts(x): pts(y) = pts(v)	pts(y) = pts(*x)

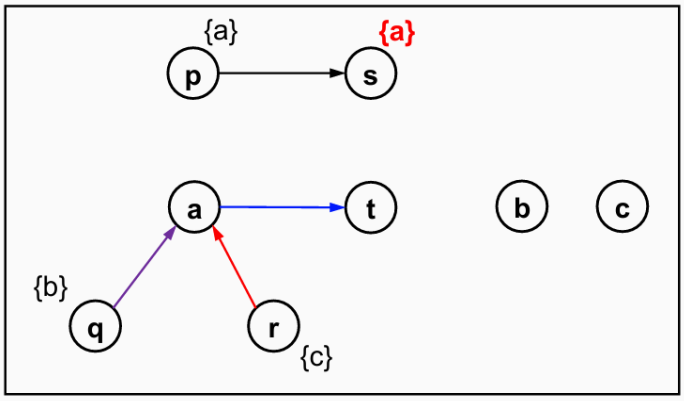
```
p  =  &a;
q  =  &b;
*p =  q;
r  =  &c;
s  =  p;
t  =  *p;
*s =  r;
```

```
pts(p) ⊇ {a};
pts(q) ⊇ {b};
...
pts(r) ⊇ {c};
pts(s) ⊇ pts(p);
...
...
```



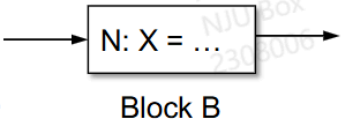

```
p  =  &a;
q  =  &b;
*p =  q;
r  =  &c;
s  =  p;
t  =  *p;
*s =  r;
```

```
...
...
pts(*p) ⊇ pts(q)
...
...
pts(t) ⊇ pts(*p)
pts(*s) ⊇ pts(r)
```

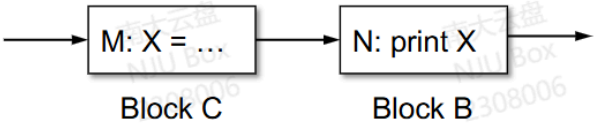


Reaching Definitions by Datalog

- def(B, N, X)
  - the Nth statement in Block B may define the variable X.



- succ(B, N, C)
  - block C is a successor of block B, and B has N statements.
- rd(B, N, C, M, X)
  - the definition of variable X at the Mth statement of block C reaches the Nth statement in B.

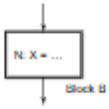


Reaching Definitions by Datalog

- rd(B, N, B, N, X) :- def(B, N, X)
- rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), def(B, N, Y), X≠Y
- rd(B, 0, C, M, X) :- rd(D, N, C, M, X), succ(D, N, B)

Reaching Definitions by Datalog

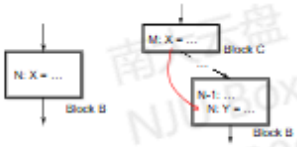
- rd(B, N, B, N, X) :- def(B, N, X)
- rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), def(B, N, Y), X≠Y
- rd(B, 0, C, M, X) :- rd(D, N, C, M, X), succ(D, N, B)



南大云盘
NJU Box
2308006

Reaching Definitions by Datalog

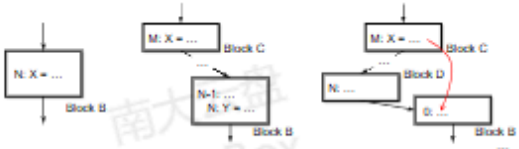
- rd(B, N, B, N, X) :- def(B, N, X)
- rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), def(B, N, Y), X≠Y
- rd(B, 0, C, M, X) :- rd(D, N, C, M, X), succ(D, N, B)



南大云盘
NJU Box
2308006

Reaching Definitions by Datalog

- rd(B, N, B, N, X) :- def(B, N, X)
- rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), def(B, N, Y), X≠Y
- rd(B, 0, C, M, X) :- rd(D, N, C, M, X), succ(D, N, B)



南大云盘
NJU Box

Reaching Definitions by Datalog

- rd(B, N, B, N, X) :- def(B, N, X)
- rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), def(B, N, Y), X≠Y
- rd(B, 0, C, M, X) :- rd(D, N, C, M, X), succ(D, N, B)

- def(B_1, 1, i)
- def(B_1, 2, j)
- def(B_1, 3, a)
- def(B_2, 1, i)
- def(B_2, 2, j)
- def(B_3, 1, a)
- def(B_4, 1, i)
- succ(B_1, 3, B_2)
- succ(B_2, 2, B_3)
- succ(B_2, 2, B_4)
- succ(B_3, 1, B_4)
- succ(B_4, 1, B_2)